

Assignment 2 – Case Study

Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	patient_id is unique and never changes; name is never blank.	get_id(), get_name(), rename(new_name) (validates), matches(query)
Practitioner	practitioner_id is unique and never changes; name is never blank.	get_id(), get_name(), appointments_on(date)
Appointment	Patient, practitioner, date and time are fixed once created; status can only change through allowed transitions (SCHEDULED to CANCELLED or COMPLETED).	cancel(), complete(), get_status(), conflicts_with(other)

Activity 2 - Composition or Inheritance?

Appointment and Patient -> ☒ Composition/association ☐ Inheritance Reason: An appointment has a patient, but it is not a kind of patient, so inheritance fails the "is a" test.

Appointment and Practitioner -> ☒ Composition/association ☐ Inheritance Reason: An appointment has a practitioner. It is not a kind of practitioner.

Doctor and Practitioner (hypothetical) -> ☐ Composition/association ☒ Inheritance Reason: A doctor is a kind of practitioner ("is a"), and inheritance only makes sense if doctors add extra behaviour.

Clinic and Appointment -> ☒ Composition/association ☐ Inheritance Reason: A clinic has appointments, and an appointment is not a kind of clinic.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

Answer: Appointment decides, because it owns its status. Its cancel() method checks the current status and rejects illegal changes.

Who validates a patient name?

Answer: Patient validates its own name, because it protects its own invariant. The UI can do basic input checks, but Patient has the final say.

Should Appointment execute SQL? Why?

Answer: No. Appointment should only hold appointment logic. SQL mixes in storage, makes testing harder and ties the class to one database. A separate storage class should do it.

Should the UI decide whether a status transition is legal?

Answer: No. The UI shows results and error messages, but the rule belongs in Appointment so every interface follows the same rule.

Activity 4 - AI Code Critique

AI generates an Appointment class with public status mutation, SQL inside cancel(), a NotificationManager dependency and inheritance from PatientRecord. Identify at least five design problems and corrections.

1. Public status mutation: anyone can set an invalid status. Correction: make status private and change it only through cancel() and complete().
2. SQL inside cancel(): this mixes business rules with storage and makes testing harder. Correction: cancel() only changes the status, and a separate storage class saves it.
3. NotificationManager dependency: no requirement asks for notifications, and it adds coupling. Correction: remove it.
4. Inheritance from PatientRecord: an appointment is not a patient record. Correction: Appointment holds a Patient (association).
5. No rule for status changes: a completed appointment could be cancelled. Correction: cancel() checks the current status and rejects illegal transitions.
6. Too many jobs in one class (low cohesion): it handles status, storage and notifications. Correction: keep Appointment focused on appointment behaviour.

Exit question

Why can code be object-oriented syntactically but still have poor object-oriented design?

Syntax only shows that classes and objects are used. Design is about where responsibilities go, whether state is protected and whether relationships make sense. A class with public fields, SQL and wrong inheritance still compiles, but it is poorly designed.